Manual Pod Scheduling

Alright, junior kube-naut 👩‍🚀
Let's do this **the senior-engineer way**: *why it exists → what breaks → how manual scheduling saves the day.*

🧭 Manual Scheduling of Pods in Kubernetes

(When the Scheduler takes a chai break ☕)

Step 0 – The "Why" (Way → Problem → Solution)

🛣️ Normal Way (Default)

- You create a Pod
- **kube-scheduler** decides **which node**
- Pod gets placed automatically

  👉 Life is good.

💥 The Problem

What if:

- Scheduler pod is **stopped / deleted**
- Control-plane scheduler crashes
- You want **full control** (for learning, debugging, or special workloads)

🚨 Result:
Pods stay in **Pending** forever.

🛠️ The Solution

👉 **Manual Scheduling**

- YOU tell Kubernetes **exactly which node** to run the Pod on
- No scheduler needed
- Old-school but powerful (like `vim` 😄)

Step 1 – Overview (High-level understanding)

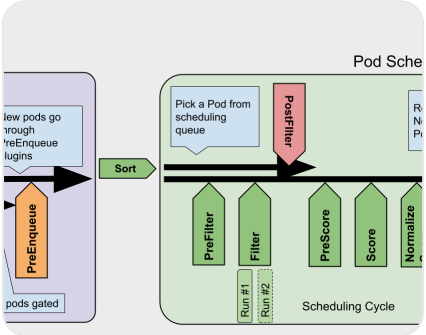
Manual Scheduling means:

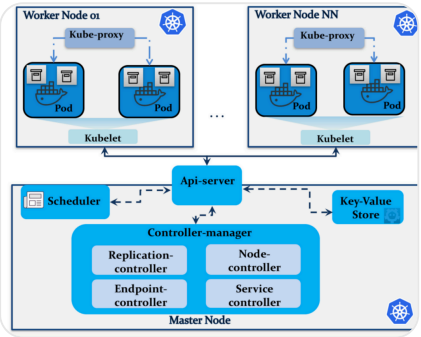
- You bypass `kube-scheduler`
- You directly assign a Pod to a node using:

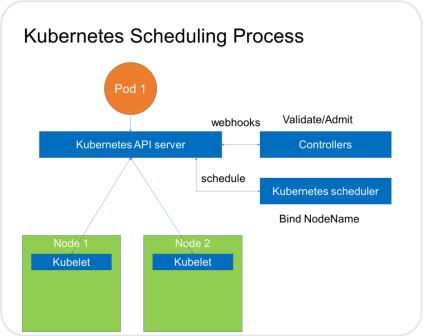
```yaml
spec:
  nodeName: <node-name>
```

📌 Kubernetes obeys you blindly here.

No decisions. No debates. Just execution.







Step 2 – Core Concepts (What's really happening)

🔑 Key Players

- **kube-scheduler**
  - Watches for Pods without a node
  - Assigns them to nodes
- **nodeName (your cheat code)**
  - If `nodeName` is set:
    - Scheduler is skipped ❌
    - Kubelet on that node picks up the Pod ✅

🧠 Golden Rule

> If `spec.nodeName` exists → scheduler is NOT required.

Step 3 – Hands-on Example (with proof 😎)

✅ Scenario

Even if we **STOP / REMOVE** the scheduler:

```bash
ls /etc/kubernetes/manifests
# kube-apiserver.yaml
# kube-controller-manager.yaml
# kube-scheduler.yaml
# etcd.yaml
```

Now imagine:

```bash
mv kube-scheduler.yaml /tmp/
```

Scheduler is DEAD 🪦

Let's still schedule a Pod.

🔍 Step 1: Get node name

```bash
kubectl get nodes
```

Example output:

```text
NAME           STATUS   ROLES
worker-node1   Ready    <none>
```

📄 Step 2: NGINX Pod YAML (Manual Scheduling)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-manual-pod
```

```yaml
    labels:
      app: nginx
  spec:
    nodeName: worker-node1   # 👈 MANUAL SCHEDULING
    containers:
    - name: nginx
      image: nginx
      ports:
      - containerPort: 80
```

Apply it:

```bash
kubectl apply -f nginx-manual-pod.yaml
```

🎉 Result

```bash
kubectl get pods -o wide
```

You'll see:

- Pod is **Running**
- Assigned to `worker-node1`
- Scheduler never involved 😎

Step 4 – Common Mistakes (a.k.a. ways to cry 😭)

❌ Wrong node name

```yaml
nodeName: worker-node999
```

➡️ Pod stuck forever in `Pending`

❌ Node is NotReady

➡️ Kubelet won't run the Pod

❌ **Forgetting scheduler is manual now**
➡️ Other Pods (without nodeName) won't schedule

❌ **Production misuse**
➡️ You lose:

- Load balancing
- Auto placement
- Scheduling intelligence

Step 5 – Advanced Usage / Optimization (Pro moves 🧠)

🔧 When is Manual Scheduling useful?

- Debugging scheduling issues
- Disaster recovery (scheduler down)
- Bare-metal clusters
- Learning internals (best reason 🔥)

⚖️ Better Alternatives (Real-world)

Instead of `nodeName`, prefer:

- `nodeSelector`
- `nodeAffinity`
- `taints & tolerations`

These:

- Still use scheduler
- But with **rules**, not dictatorship 😄

🧠 Memory Trick (Exam + Interview Gold)

> **"No scheduler? No problem. nodeName decides destiny."**

If you want, next we can go deeper into:

- `nodeSelector vs nodeName`
- `How kubelet creates Pod without scheduler`

- OR a **scheduler failure troubleshooting scenario (interview-style)**

Say the word, senior-in-training 😄 👊